LOF CRM Technical Specification

Oct 12, 2026 · @Mushtaq

Lab of Future CRM runs as one Cloudflare Worker with a D1 (SQLite) database, a job queue and a 5-minute scheduler; the approved prototype screens talk to it through a REST API and a permission-guarded sync endpoint.

Scope and system context

The CRM manages Lab of Future's admissions cycle across 5 branches in two currencies: Al Karama, Sports City and Abu Dhabi (AED) and Bengaluru and Hyderabad (INR).

Function	Covered
Lead management	Capture from website, Meta Lead Ads, WhatsApp, phone and manual entry; duplicate detection; automatic routing; first-call SLA; follow-ups
Sales pipeline	Opportunities, admission decisions, discount requests and approvals
Admissions	Student registration, enrolments with server-computed fees and payment terms (instalments), LMS enrol and cancel calls
Payments	Counter payments, hosted payment links (Telr for AED, Razorpay for INR), receipts, refunds, ERPNext posting
Communications	WhatsApp and e-mail with consent checks; telephony via Exotel
Governance	Roles and permissions matrix, branch scope, approvals, append-only audit trail, reports
Administration	Configuration centre, staff and logins, Testing console

Users: 6 roles – Counsellor, Centre Administrator, Admissions Head, Reception, Management, System Administrator. Each user is limited to their branches. External systems: Meta, WhatsApp Cloud API, Exotel, the LMS (LOF booking platform), Telr, Razorpay, ERPNext, Google Calendar and Resend e-mail.

Architecture

Everything runs inside one Cloudflare Worker; there are no servers to manage.

Component	Technology	Responsibility
CRM screens	The approved prototype (single HTML page) + public/api-client.js	All screens and workflows; sends changes to the server and reloads server state
Static hosting	Workers static assets with a generated _headers file	Serves the screens with CSP script hashes and security headers
API	Hono on Cloudflare Workers	/api/v1 REST, sync, auth and Testing console; /webhooks for providers; /health
Database	Cloudflare D1 (SQLite)	Documents with indexed columns, settings, sequences, sessions, logs; triggers and reporting views
Job queue	Cloudflare Queues (lof-crm-jobs + dead-letter queue)	LMS enrol/cancel calls, WhatsApp and e-mail sends, ERPNext posting, calendar events; 3 retries
Scheduler	Cron trigger every 5 minutes	SLA escalation, overdue follow-ups and instalments, LMS and ERPNext retries, Telr reconciliation, data retention
Integration adapters	One module per provider, live or mock	Outbound calls with timeouts and redacted logs

Request flow for a change made on a screen:

1. The user acts on a screen; the prototype's workflow code updates its in-memory state.
2. api-client.js diffs the state against the last server copy and sends the changed records, each with its base version, to PUT /api/v1/sync.
3. The Worker authenticates the session, checks the CSRF header and rate limits.
4. The sync engine validates every change (role, branch, approvals, fees, money, consent, history), merges field by field and writes all of them or none.
5. Accepted changes produce jobs (messages, LMS calls, ERPNext, calendar) that go to the queue; the response returns the server versions and notices.
6. The screen reloads server state every 45 seconds and on demand; incoming calls are polled every 8 seconds.

Technology stack and code structure

Item	Choice
Runtime	Cloudflare Workers (compatibility date 2026-10-01, nodejs_compat)
Language	TypeScript (strict)
Web framework	Hono 4.13
Database	Cloudflare D1 (SQLite), 4 migrations
Crypto	Web Crypto: PBKDF2-SHA256, HMAC-SHA256, AES-GCM, SHA-256, RS256 (Google service account)
Tooling	Wrangler 4, Vitest 3, Playwright, Node 22
Frontend	Prototype HTML + api-client.js; QR code generator 1.4.4 (MIT) for two-step set-up

Path	Contents
src/index.ts	Worker entry: security headers, CORS, routes, cron and queue handlers
src/routes/	auth.ts, api.ts (REST + sync), webhooks.ts, testing.ts, receipt.ts
src/sync/	bootstrap.ts (scoped state), sync.ts (guarded writes)
src/domain/	core.ts (fees, routing, duplicates, leads, payments), workflows.ts (LMS calls, messages, ERPNext, payment links), terms.ts (instalments, refund rules, targets, LMS payload)
src/integrations/	base.ts (mode, logging, redaction) and one adapter per provider
src/lib/	db.ts (repository), indexing.ts, auth.ts, session.ts, totp.ts, crypto.ts, ratelimit.ts
src/jobs.ts	Job runner, queue consumer, scheduler
migrations/	0001 schema, 0002 seed (generated), 0003 reporting views, 0004 security

Path	Contents
scripts/	patch-frontend, build-seed, create-admin, cf-deploy
public/	index.html (generated), api-client.js, embed/lead-form.js, vendor/
test/, e2e/	Vitest suites and Playwright browser tests

Data model

Business records use a document-and-index pattern: each record is stored as the same JSON document the screens use, with indexed columns derived on every write for scoping and lookups.

Tables

Table	Key	Purpose
docs	(type, id)	All business records: JSON data + branch, status, owner, ref1, ref2, at, is_test, version, created/updated stamps
settings	key	Single documents: cfg (configuration centre), perms (role matrix), scope (multi-branch access), fx (report FX rates), hist (admissions history), rr_last (round-robin pointer)
sequences	name	Next number for each series; receipt series per branch (rc_KRM, rc_BLR …)
staff_auth	staff_id	Password hash, must-change flag, failed count, lock, last login, encrypted TOTP secret, two-step state
sessions	id_hash	SHA-256 of the session token, staff, created, last seen, expiry, revoked, stage, acting-as, IP, user agent
rate_limits	key + window	Fixed-window counters for sign-in, two-step, sequences, website form, per-user API

Table	Key	Purpose
integration_logs	id	Every outbound call and inbound webhook, redacted; 30-day retention
webhook_events	(provider, event_id)	Idempotency for provider retries; 90-day retention
payment_links	id (PL- + 10 random characters)	Telr/Razorpay link, amount, currency, status, payment once paid
call_events	id	Incoming-call screen-pop events; 2-day retention

Document types and index columns

Type (collection)	ID format	branch	ref1	ref2	owner
lead (leads)	LD-26-####	lead branch	normalised phone (last 9 digits)	lower-case e-mail	counsellor
opp (opps)	OP-26-####	branch	lead id	student id	counsellor
student (students)	LOF-26-######	home branch	–	–	–
enrollment (enr)	EN-26-####	branch	student id	opportunity id	–
payment (pay)	PY-26-####, receipt RC-<branch>-26-####	branch	enrolment id	student id	–
refund (refunds)	RF-26-####	branch	enrolment id	–	–
moodle_sync (ms) – LMS calls	MS-####	branch	enrolment id	student id	–
approval (approvals)	AP-26-####	branch	referenced record	–	–
task, call, comm	T-, CL-, CM-	lead branch	lead id	–	owner
notification, audit	N, AU	branch	–	–	–
branch, program, fee, discount, staff	master ids	–	staff: lower-case e-mail	–	–

Test records carry is_test = 1 and TST- ids, never consume business numbers, and are visible only to System Administrators.

Numbering

Gapless series (lead, opp, enr, pay, student, ap, rf, pd) are claimed one at a time with /seq/claim; internal series (task, call, cm, notif, audit, ms) are handed out in blocks with /seq/block. Receipt numbers are assigned by the server only after a payment is validated. A new record whose number was not issued by the server is refused.

Database triggers

Trigger	Rule
trg_payment_no_overpay	Refuses a new payment that would take an enrolment's paid total above its total
trg_payment_immutable	Payment amount, enrolment, currency and receipt number cannot change
trg_payment_no_delete	Payments cannot be deleted (test records excepted)
trg_audit_no_update / trg_audit_no_delete	Audit entries cannot be changed or deleted (test records excepted)

Reporting views

v_leads, v_opportunities, v_students, v_enrollments, v_payments and v_moodle_syncs (migration 0003) and, from migration 0005, v_instalments (one row per instalment, via json_each over the schedule), v_refunds, v_lms_calls, v_approvals, v_tasks, v_calls, v_staff (sign-in state joined from staff_auth, never the password hash or two-step secret), v_targets (one row per counsellor-month from settings) and v_audit flatten the JSON documents into columns for SQL and BI tools. 0005 also adds student e-mail/mobile and family to v_students, and counsellor, payment term and outstanding amount to v_enrollments.

Database browser, export and backup

Settings → Database (System Administrator only) is a read-only browser over 19 whitelisted datasets: the business views above plus integration_logs, payment_links, webhook_events, sessions (metadata only, never the token hash) and sequences. Each dataset maps to fixed SQL with an explicit column list, so no request value reaches a table or column name; search is a parameterised LIKE across the listed columns, with branch and test-record filters and paging (⩽ 500 rows).

Endpoint	Purpose	Limits
GET /api/v1/admin/db/tables	Datasets with row counts	–
GET /api/v1/admin/db/rows	Filtered, paged rows	limit ⩽ 500
GET /api/v1/admin/db/rows.csv	CSV of the current filter; BOM; cells starting = + - @ prefixed with '	50,000 rows, 30/hour, audited
GET /api/v1/admin/db/record	Full stored record with version and last change	–
GET /api/v1/admin/db/backup	lof-crm-backup/1 JSON: all docs, settings, sequences (no logins, sessions or logs)	5/hour, audited

scripts/restore-backup.mjs turns a backup into SQL for wrangler d1 execute: records in dependency order with index columns recomputed, payments and audit entries INSERT OR IGNORE (the database forbids changing them), settings replaced, sequences raised with MAX so a restored number is never issued again.

Roles, permissions and branch scope

Permissions come from the Roles & Permissions matrix (settings.perms), editable by configuration roles. Capabilities: V view, C create, E edit, A approve, D delete. Default matrix:

Resource	Counsellor	Centre Admin	Admissions Head	Reception	Management	System Admin
Leads	VCE	VCE	VCE	V	V	VCED
Opportunities	VCE	VCE	VE	V	V	VCED
Enrolments	V	VCE	VC	VC	V	VCED
Students	V	VE	V	VE	V	VCED
Payments	–	V	V	VC	V	VCE
Discount requests	VC	VC	VC	V	VA	V
Branch transfers	VC	VC	VA	–	V	V
Reassignment	C	VCE	VAE	–	V	V

Branch scope: each user sees and changes only records of their branch (or the branches granted in settings.scope; Admissions Heads cover their country; Management and System Administrator cover all). Scope is checked on reads, on the stored version and on the new version of every write.

Special rules: the Testing console and staff password administration are System Administrator only; the System Administrator always keeps Configuration view and edit; role preview is disabled in production.

Guarded sync

PUT /api/v1/sync is the single write path for the screens. A call carries up to 500 changes (2 MB, 64 KB per record) plus setting changes, and is applied all-or-nothing.

1. Input checks: only upsert and delete; ids match a safe pattern and equal the document id; no duplicate changes; known collections only.
2. Issued numbers: a new business record must use a number below the server's current sequence value.
3. Field-level merge: the server compares the client's base version with its new version to find the fields this user changed, and applies only those to the current stored record. A colleague's concurrent change to other fields is kept; a clash on the same field returns a notice.
4. History lists (events, assignments, approval history, messages, testimonials, documents, WhatsApp events) can only be appended to; appended entries are stamped with the server time and the real user.
5. Rules per record type (next section). Related records are always read as stored, never from unchecked changes in the same batch, and records are checked in dependency order (masters, leads, opportunities, approvals, students, enrolments, payments, LMS calls, refunds, tasks, calls, messages, notifications, audit).
6. Apply: server-owned fields are set (receipt numbers, stamps, test flags), writes run in one database batch, triggers give a last check, follow-on updates run (counsellor moves update open tasks and opportunities; branch moves update child records), enrolment balances are recalculated.
7. Jobs for messages, LMS calls, ERPNext and calendar are queued; the response returns the stored documents and notices.

Any rejection returns 409 with the reasons and nothing is written.

Business rules by module

Module	Rules enforced on the server
Leads	Required student, parent, phone, branch, programme, source; branch must offer the programme; source must be enabled; duplicate check on normalised phone and e-mail (other branches shown masked); routing by load or round-robin among active counsellors handling the programme with free capacity, else Unassigned queue; a branch change needs an approved transfer for exactly that branch; a counsellor change needs reassignment rights or a matching approval; users without lead edit rights may only log calls, follow-ups and availability
Opportunities	Built from the lead (branch, programme, counsellor); package must have an active fee; Enrolled only through the enrolment form; exceptional discount only with an approved request for that opportunity and value
Approvals	Built on the server: approver role from configuration, validated payload, server summary and history; decided only by the configured role, never by the requester; a decided request cannot change
Enrolments	Student and opportunity in scope; date limited to the last 7 days; fee recomputed from the fee master (programme fee + registration for new students − discount, then tax on the net); discount is none, a predefined active discount, or an approved exceptional discount for the same opportunity, programme and fee, usable once; payment term must be active (default if none); schedule and credited counsellor set by the server and fixed; a withdrawal cannot be reversed

Module	Rules enforced on the server
Payments	Payments create rights, or enrolment create rights for the first payment of a new enrolment; amount ⩽ outstanding; method from configuration; reference for non-cash; date within the last 30 days; receipt number from the branch series; immutable after saving; posted to ERPNext
Payment links	Telr for AED branches, Razorpay for INR; amount ⩽ outstanding; random id; payment recorded only after the provider confirms; a link is settled once
Refunds	Sessions attended required; suggestion computed from the refund rules (what remains after earlier requests); a request above it is an exception that only the approver role, and not the requester, can process or reject; cumulative total ⩽ amount paid; Requested → Processed or Rejected
Students	Id, registration, branch and family links fixed after creation; the welcome e-mail is built on the server and sent to the stored address
LMS calls	One enrol call per new enrolment and one cancel call per withdrawal (created by the server if the client omits it); Track and Credit codes taken from settings; never sent with missing required fields; an enrol call is never sent after a withdrawal; a successful call cannot be re-run; retries by user or scheduler
Communications	Sent only with WhatsApp or e-mail consent, otherwise stored as Blocked; client-written text on students is recorded, not sent
Settings	Configuration roles only; the role matrix is validated; secrets never stored in settings

Counsellor targets, payment terms, refund rules and LMS (version 2)

All four live in the configuration document (settings.cfg) and are edited under Settings by configuration roles; the server validates every save and recomputes everything that affects money or access.

Feature	Stored in cfg	Server rules	Shown in
Counsellor targets	targets: [{ m: YYYY-MM, c: counsellor id, enr, rev, conv }]	Month format, existing counsellor, whole enrolments, revenue ≥ 0, conversion 0–100%, one row per counsellor and month; counsellors receive only their own rows	Settings → Counsellor Targets (copy previous month, add next month, comparison with branch targets); My monthly target tile; report 21; GET /targets
Payment terms	payTerms: [{ id, name, split[%], gap days, minFirst %, grace days, active }], payTermDefault	Split adds to 100, 1–12 instalments, first instalment ≥ minimum, default exists and is active; enrolment schedule (sched) computed on the server from the term, total and date	Enrolment form (term + schedule), Payment Collection, Enrolments list, Reception tile Overdue instalments, report 22; scheduler notifies once per overdue instalment
Refund rules	refund: { rules[{ label, days, sessions, pct }], regFee, tax, adminFee{AED, INR}, approver, confirmed }	First rule where days since enrolment ⩽ days and sessions attended ⩽ sessions applies; refund = programme fee after discount × pct + tax share (if refunded) + registration (if refunded) − admin fee, capped at amount paid; earlier requests use up the suggestion	Withdraw form (live suggestion, exception warning), Refunds page, Settings calculator, GET /refunds/suggest

Feature	Stored in cfg	Server rules	Shown in
LMS integration	lms: { validMonths, programmes{ id: { track, credits[] } }, branchMap{ branch: code }, confirmed }, integ.lms: { on, retryMax, retryMin, notify }	Codes: letters, digits, . _ - only, no duplicates; endpoint (LMS_URL) and token (LMS_TOKEN) are server secrets; payload built on the server; valid_until = enrolment date + validMonths − 1 day (clamped at month end)	Settings → LMS Integration (Calls, Setup with code table, branch map, field mapping and readiness checklist, Sample data); Integration Monitor; reports 17–18; GET /lms/preview/:id

Instalment status: payments are applied to the earliest instalment first; an instalment is Paid, Partly paid, Due (on or after its due date), Upcoming, Overdue (unpaid after due date + grace days) or Closed (enrolment withdrawn or completed).

LMS calls follow “LOF Booking Platform – what ERPNext sends us” (v3, 19 Sep 2026): enrol = reference (enrolment id), student_id (LOF student id), names, e-mail and phone (parent’s when the student has none), date of birth, parent_id (family id), parent name/e-mail/phone, branch code, valid_until, and per product the product name, Track code and exact Credit codes; cancel = reference only. No amount, price, invoice, payment method or refund value is sent. Placeholders to confirm: refund rule values and LOF’s Track/Credit code list.

Authentication and sessions

Control	Specification
Password storage	PBKDF2-SHA256, 100,000 iterations, 16-byte random salt
Password policy	⩾ 10 characters, 3 of 4 classes (lower, upper, digit, symbol), not containing the e-mail name or common words; must change at first sign-in and after a reset
Lockout	5 failures lock the account for 15 minutes; one generic error message; unknown e-mails take the same time

Control	Specification
Rate limits	Sign-in 10 per e-mail and 30 per IP per 15 min; two-step 10 per 10 min; password change 10 per 15 min; API 600 per user per minute
Two-step verification	TOTP (RFC 6238, 30-second step, ±1 step), secret AES-GCM encrypted with a key derived from JWT_SECRET, replay blocked by last used step; required for MFA_REQUIRED_ROLES
Session token	256-bit random; only its SHA-256 hash is stored
Browser transport	__Host-lof_session cookie: HttpOnly, Secure, SameSite=Strict, Path=/; writes require header x-lof-csrf: 1
API clients	Bearer token returned by sign-in when the CSRF header is absent
Lifetime	60 minutes idle (SESSION_IDLE_MINUTES), 12 hours absolute
Revocation	Logout; password change and two-step changes end other sessions; administrator reset or Sign out everywhere ends all
Stages	password_change and mfa_setup sessions may call only /auth/*
Sign-in endpoint	JSON only; a browser Origin must be the app's own origin

Integrations

An integration runs live when it is enabled under Configuration → Integrations and all its secrets are set; otherwise it runs in mock mode with realistic responses. INTEGRATION_MODE=mock forces mock everywhere; test records always run in mock mode in production. Every call is logged with secrets redacted, with a 20-second timeout.

Integration	Direction	Operations	Inbound verification
Website form	In	Lead capture (embed script or signed POST)	HMAC-SHA256 signature or allowed Origin; honeypot; rate limit; optional Turnstile

Integration	Direction	Operations	Inbound verification
Meta Lead Ads	In	Leadgen webhook → Graph API fetch → lead	Verify token; X-Hub-Signature-256; event de-duplication
WhatsApp Cloud API	Out + in	Template and text messages; delivery status; inbound replies onto the lead	Verify token; X-Hub-Signature-256
Exotel	Out + in	Click-to-call (counsellor first, then parent); incoming-call screen-pop; call logging	Shared token; optional IP allow-list
LMS (LOF booking platform)	Out	Enrol call (student, parent, branch, valid_until, Track + exact Credit codes per product) and cancel call (reference only); JSON with a bearer token to one endpoint; no payment data	–
Telr (AED)	Out + in	Hosted payment order; check API; transaction advice	Every payment confirmed with Telr's check API; scheduler reconciles
Razorpay (INR)	Out + in	Payment links; paid, cancelled, expired events	X-Razorpay-Signature; event id de-duplication
ERPNext	Out	Customer and draft Payment Entry per payment	–
Google Calendar	Out	Event per scheduled call (service account, RS256)	–
Resend e-mail	Out	Transactional e-mail	–

Background jobs and scheduler

Job (queue)	Trigger	Behaviour on failure
moodle_sync	New enrolment; manual retry	Error stored on the sync; scheduler retries; escalates after the retry limit

Job (queue)	Trigger	Behaviour on failure
send_comm	Accepted WhatsApp or e-mail message	Status Failed with the provider error on the lead timeline
send_student_mail	Welcome e-mail on a new student	Status Failed on the student record
erp_payment	Each recorded payment	Error kept on the payment; scheduler retries up to 5 times
calendar_event	Scheduled call task	Logged; task unaffected

Queue: 10 messages per batch, 3 retries with 60-second delay, then the dead-letter queue. Without a queue binding (local runs) jobs run in the background of the request.

Scheduler, every 5 minutes: first-call SLA alerts and escalation; overdue follow-up notifications (once each); overdue instalment notifications to Reception (once per instalment, after the grace days); LMS auto-retry; ERPNext retry; Telr reconciliation of open orders; data retention (integration logs 30 days, webhook ids 90 days, ended sessions, rate counters, call events after 2 days).

Configuration

Variables (wrangler.toml)

Variable	Default	Purpose
APP_ENV	production	production, staging or test; production disables role preview and forces test records to mock
APP_BASE_URL	–	Public URL; webhook and payment return URLs are built from it
INTEGRATION_MODE	auto	auto (live when enabled and configured) or mock
ALLOW_IMPERSONATION	false	Role preview for System Administrators (never in production)
MFA_REQUIRED_ROLES	System Administrator, Management	Roles that must use two-step verification
SESSION_IDLE_MINUTES	60	Idle session timeout

Variable	Default	Purpose
EXOTEL_ALLOWED_IPS	empty	Optional allow-list for Exotel webhooks
Provider variables	–	META_GRAPH_VERSION, WHATSAPP_PHONE_NUMBER_ID, WHATSAPP_TEMPLATE_LANG, EXOTEL_SUBDOMAIN, EXOTEL_SID, EXOTEL_CALLER_ID, LMS_URL, TELR_STORE_ID, TELR_TEST_MODE, RAZORPAY_KEY_ID, ERPNEXT_URL, ERPNEXT_COMPANY, ERPNEXT_PAID_TO_AED, ERPNEXT_PAID_TO_INR, GCAL_CALENDAR_ID, EMAIL_FROM

Secrets (wrangler secret put)

Secret	Used for
JWT_SECRET	Required, ⩾ 32 characters; derives the two-step encryption key and webhook test tokens. Rotating it disables everyone's two-step
TURNSTILE_SECRET	Optional bot check on the website form
WEBSITE_FORM_SECRET	Website form signatures
META_APP_SECRET, META_VERIFY_TOKEN, META_PAGE_ACCESS_TOKEN	Meta Lead Ads
WHATSAPP_ACCESS_TOKEN, WHATSAPP_APP_SECRET, WHATSAPP_VERIFY_TOKEN	WhatsApp
EXOTEL_API_KEY, EXOTEL_API_TOKEN, EXOTEL_WEBHOOK_TOKEN	Exotel
LMS_TOKEN	LMS bearer token (enrol and cancel calls)
TELR_AUTH_KEY	Telr
RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	Razorpay
ERPNEXT_API_KEY, ERPNEXT_API_SECRET	ERPNext

Secret	Used for
GCAL_SERVICE_ACCOUNT_JSON	Google Calendar
RESEND_API_KEY	E-mail

Business configuration (branches, programmes, fees, discounts, staff, routing, SLA, templates, payment methods, notification rules, integration toggles) is edited in the CRM's Configuration centre and stored in settings.cfg; credentials are never stored there.

Non-functional requirements

Area	Specification
Security headers	CSP with SHA-256 hashes for inline scripts, connect-src 'self', frame-ancestors 'none'; API default-src 'none'; HSTS 1 year; X-Frame-Options DENY; nosniff; strict Referrer-Policy; Permissions-Policy; COOP; no-store on API responses
CORS	Only the app's own origin, with credentials
Data integrity	All-or-nothing saves; database triggers on payments and audit; append-only history; server-issued numbers
Least data	Password hashes and secrets never leave the server; staff contact details only for configuration roles; payments only with Payments → View
Limits	500 changes and 2 MB per save; 64 KB per record; 200 items per list page; 20-second provider timeout
Retention	Integration logs 30 days; webhook ids 90 days; call events 2 days; ended sessions and expired rate counters removed
Availability	Cloudflare global edge; D1 Time Travel point-in-time restore (30 days); Worker rollback to any previous version
Observability	/health endpoint; integration log viewer in the Testing console; Worker logs in Cloudflare; dead-letter queue for failed jobs

Area	Specification
Time and currency	Business time zone Asia/Dubai; AED and INR kept separate; reporting conversion through a manual FX setting
Build size	290 KiB (75 KiB gzipped)
Browser support	Current Chrome, Edge, Safari and Firefox; usable at phone width

Environments and deployment

Environment	Worker	Database	Integrations	Data
Local	wrangler dev	Local D1	Mock	Demo data
Trial (Claude)	Static page in Claude	None – browser memory	Simulated	Demo data, per viewer, resets on reload
Staging	lof-crm-staging	lof_crm_staging	Forced mock; role preview on	Demo data
Production	lof-crm	lof_crm	Live per integration once configured	Masters only; real staff

Deployment is one command, scripts/cf-deploy.mjs, run from a terminal or the GitHub workflow (.github/workflows/deploy.yml, which runs the type check and all tests first). It creates the database and queues if missing, applies only new migrations, deploys, sets JWT_SECRET only when absent, creates the first administrator only when it does not exist, and checks /health. Rollback: wrangler rollback for code; D1 Time Travel for data. deploy.sh (macOS/Linux) and deploy.ps1 (Windows) wrap it: they prompt for the Cloudflare account ID, API token and administrator details (password typed hidden, only its hash stored), run the type check and tests, then deploy. The step-by-step guide is DEPLOY.md in the code package; the runbook is in LOF CRM – Test, Security & API Handover.